

Report tesi

Report di avanzamento tesi - aggiornamento al 27 agosto 2026

1. Obiettivo e perimetro del lavoro

Questo report ricostruisce il lavoro svolto dopo il report di inizio agosto 2026. Il punto di arrivo della fase precedente era un insieme di artefatti più robusto, un Training set supervisionato, un primo Dataset per l'LLM con vincoli espliciti di provenienza e validazione e lo scheletro del prototipo.

Durante questa fase mi sono reso conto che il lavoro si stava allontanando dall'obiettivo principale della tesi: costruire un modello che, ricevuti un circuito quantistico e una figure of merit, consigli il dispositivo più adatto e la relativa configurazione di Qiskit.

Ho quindi rivisto la struttura del Dataset affinché ogni esempio contenga il circuito logico, la figure of merit, il dispositivo migliore, la configurazione migliore e una motivazione sostenuta da prove verificabili. In parallelo ho definito i vincoli hardware dell'utente, un primo criterio di somiglianza tra circuiti e la politica da usare quando l'LLM produce una risposta non valida.

2. Stato iniziale: da dove sono partito

2.1 Un Dataset adatto all'obiettivo

La struttura precedente conservava anche le tracce dei modelli RL e le informazioni legate ai modelli supervisionati. Questi dati erano interessanti, ma non aiutavano direttamente a rispondere alla domanda principale della tesi.

Ho quindi creato il ramo di lavoro `qiskit_dataset`, nel quale ho mantenuto soltanto ciò che serve per costruire il nuovo Dataset. Il ramo `main` conserva invece il lavoro precedente e rimane disponibile nel caso in cui diventi utile per sviluppi futuri.

Il nuovo Dataset deve descrivere come dovrebbe essere una buona risposta dell'LLM. Ogni esempio comprende quindi il circuito logico, la figure of merit, il dispositivo migliore, la configurazione migliore e una motivazione con i relativi riferimenti, cioè i campi `claim` ed `evidence`.

Per costruire questi esempi è stato necessario definire formalmente che cosa si intende per configurazione di Qiskit. I parametri disponibili in `qiskit.transpile` sono numerosi; in questa prima fase mi sono concentrato sul dispositivo, sul `layout_method`, sul `routing_method` e sull'`optimization_level`. Ho scelto questi parametri perché influenzano alcune delle fasi più importanti e delicate della compilazione.

2.2 Struttura e validazione dei vincoli dell'utente

All'inizio i vincoli dell'utente non erano ancora definiti. Una possibilità era permettere di descriverli in linguaggio naturale, ma questo avrebbe aumentato la complessità di un aspetto secondario del sistema.

Inoltre, non tutti i vincoli possono essere controllati in modo affidabile. Ad esempio, è difficile garantire in anticipo una particolare struttura del circuito compilato oppure la durata della compilazione.

I vincoli più semplici da descrivere e verificare sono quelli legati all'hardware. Ho quindi deciso di applicare vincoli controllati al catalogo dei dispositivi. L'utente può indicare i provider ammessi, i

dispositivi ammessi, il numero minimo e massimo di qubit e i gate nativi richiesti. La struttura scelta viene descritta più precisamente nella sezione 3.2.

2.3 Somiglianza tra circuiti

Per recuperare dal Dataset gli esempi più utili è necessario definire che cosa significa che due circuiti sono simili. Non è sufficiente avere molti esempi, perché la finestra di contesto dell'LLM è limitata e bisogna selezionare soltanto quelli più pertinenti.

La prima soluzione consiste nel confrontare i vettori delle caratteristiche dei circuiti, i feature vector. I circuiti con la distanza minore vengono considerati i candidati più simili. Questo metodo è semplice, ma parte dall'ipotesi che una somiglianza numerica tra le caratteristiche corrisponda anche a un comportamento simile durante la compilazione. Questa ipotesi non è stata ancora verificata sperimentalmente.

Come possibile sviluppo futuro ho considerato anche metodi basati sulla rappresentazione dei circuiti come grafi aciclici diretti (DAG). Tra le possibilità ci sono il Weisfeiler-Lehman Graph Kernel, più adatto a grafi grandi, e la Graph Edit Distance, più costosa e quindi utilizzabile soprattutto con circuiti piccoli.

Per ora utilizzo la distanza tra i vettori delle caratteristiche come primo criterio di somiglianza.

2.4 Validazione della risposta e nuovi tentativi

Prima di definire la politica di retry è necessario stabilire quando una risposta dell'LLM è valida. La risposta contiene tre parti: il dispositivo consigliato, la configurazione di Qiskit e la motivazione con le relative prove.

Il dispositivo è valido se appartiene al catalogo e rispetta i vincoli hardware dell'utente. La configurazione è valida se usa soltanto i valori ammessi. Questi controlli sono semplici e deterministici.

La parte più difficile da verificare è la motivazione. Gli esempi del Dataset contengono informazioni oggettive, perché tutte le combinazioni considerate sono state compilate e confrontate. Per un circuito nuovo, invece, l'LLM deve basare la propria scelta sui casi storici più simili recuperati dal Dataset.

Una risposta ideale potrebbe, ad esempio, raccomandare `quantinuum_h2_56` perché è risultato il dispositivo migliore in sei degli otto casi più simili e presenta il miglior rank medio pesato. La configurazione proposta potrebbe essere motivata dal fatto che è risultata la migliore nel 68% pesato dei casi recuperati. La risposta deve anche chiarire che il nuovo circuito non è stato ancora compilato e che il consiglio è una previsione basata su casi analoghi.

Le statistiche devono essere calcolate a partire dai dati recuperati e fornite all'LLM. Il modello non deve inventarle basandosi soltanto sulla somiglianza tra i circuiti.

3. Che cosa ho deciso di fare

3.1 Struttura del Dataset

Ogni esecuzione è definita dalla combinazione di circuito, figure of merit, dispositivo, configurazione di Qiskit e seed.

Per questa prima versione ho fissato le seguenti scelte:

- una sola figure of merit, `expected_fidelity`; in futuro potrà essere aggiunta anche `critical_depth`;
- cinque dispositivi: `ibm_falcon_27`, `ibm_falcon_127`, `ibm_heron_133`, `ibm_heron_156` e `quantinuum_h2_56`;
- quattro valori per `layout_method`: `default`, `dense`, `sabre` e `trivial`;
- cinque valori per `routing_method`: `default`, `basic`, `lookahead`, `sabre` e `none`;
- quattro valori per `optimization_level`: 0, 1, 2 e 3;
- tre valori diversi del seme casuale.

Ho scelto questi cinque dispositivi perché sono gli stessi per i quali sono già disponibili i modelli RL necessari a un futuro confronto con MQT Predictor.

La configurazione iniziale produceva 1.200 compilazioni per circuito:

$$5 \times (4 \times 5 \times 4) \times 3 = 1.200$$

Con 600 circuiti sarebbero state necessarie 720.000 compilazioni.

Per ridurre il numero di combinazioni ho escluso i livelli di ottimizzazione 0 e 1, poco adatti quando l'obiettivo principale è la qualità del circuito. Ho escluso anche il valore `none` del metodo di routing, perché causa il fallimento dei circuiti che richiedono questa fase.

In questo modo il numero è sceso a 480 compilazioni per circuito e a 288.000 compilazioni complessive.

La struttura completa del Dataset e il compito dei relativi file sono descritti in `datasets/expected_fidelity/README.md`.

3.2 Vincoli hardware dell'utente

Ho scelto di rappresentare i vincoli in forma controllata e non in linguaggio naturale. L'utente può specificare:

- i provider ammessi;
- i dispositivi ammessi;
- il numero minimo e massimo di qubit;
- i gate nativi che devono essere presenti.

Questi vincoli funzionano come filtri applicati al catalogo hardware. Sono semplici da verificare e permettono di scartare i dispositivi incompatibili prima di interrogare l'LLM.

La richiesta dell'utente viene rappresentata come un oggetto JSON contenente il circuito, la figure of merit e i vincoli hardware. Questa scelta è adatta alla struttura modulare del prototipo, perché consente di trasferire la richiesta tra i diversi moduli in modo semplice.

I dettagli della richiesta sono descritti in `prototype/README.md`.

3.3 Somiglianza tra circuiti

La somiglianza è attualmente calcolata confrontando i vettori delle caratteristiche dei circuiti. È una prima soluzione semplice e utilizzabile, ma non è ancora stato dimostrato che circuiti vicini secondo queste caratteristiche si comportino allo stesso modo durante la compilazione.

Due circuiti con vettori simili possono infatti avere DAG differenti. Questo limite dovrà essere verificato durante gli esperimenti e potrà portare all'uso di metodi più adatti alla struttura dei circuiti.

3.4 Validazione della risposta e politica dei nuovi tentativi

La validazione del dispositivo e della configurazione può riutilizzare il catalogo e gli stessi controlli applicati alla richiesta dell'utente.

Una risposta formalmente valida deve usare un dispositivo ammesso, una configurazione valida e una motivazione fondata sui casi storici recuperati. La qualità della motivazione verrà controllata verificando che faccia riferimento a esempi e statistiche realmente presenti nei dati forniti al modello.

Se la risposta non supera questi controlli, il modello potrà ricevere una nuova richiesta contenente l'errore rilevato. La validazione dell'uscita e questa politica dei nuovi tentativi non sono ancora state implementate.

4. Quali test ho eseguito

Le prove automatiche sono raccolte nella cartella tests/.

In questa fase sono state implementate e provate due parti principali. La gestione dei vincoli hardware e la validazione della richiesta sono controllate da tests/test_request_constraints.py.

La generazione e l'aggregazione del Dataset di Qiskit sono invece controllate da tests/test_qiskit_dataset.py e tests/test_qiskit_dataset_aggregation.py.

Prove sperimentali più complete saranno eseguite quando il Dataset sarà stato generato per tutti i 600 circuiti.

5. Quali risultati ho ottenuto

5.1 Dataset

Ho popolato il Dataset con dieci circuiti di esempio per verificare il processo di generazione e la struttura dei dati. I risultati di questa prima prova si trovano in datasets/expected_fidelity/pilot/, in particolare nei file JSON, .csv e .md.

La generazione usa i moduli presenti in qiskit_dataset/, richiamati dagli script 07, 08, 09 e 10 della cartella scripts/.

Questa prima prova non serve ancora a valutare la qualità dei consigli. Il suo scopo è verificare che le combinazioni vengano generate, registrate e aggregate nel formato previsto.

5.2 Prototipo LLM

Il prototipo è stato esteso con la costruzione e la validazione della richiesta dell'utente. La richiesta contiene il circuito, la figure of merit e i vincoli hardware espressi in forma strutturata.

Questa parte permette di controllare l'ingresso prima di passarlo ai moduli successivi del prototipo.

6. Prossimi passi

Il primo passo sarà completare la generazione del Dataset per tutti i 600 circuiti e svolgere prove più complete sui risultati ottenuti.

Successivamente sarà necessario verificare se la distanza tra i vettori delle caratteristiche seleziona davvero circuiti con un comportamento simile. Se questo criterio non sarà sufficiente, potranno essere valutati anche metodi basati sui grafi dei circuiti.

Resta inoltre da implementare la validazione della risposta dell'LLM e la politica dei nuovi tentativi. Questa parte dovrà controllare il dispositivo, la configurazione e la coerenza della motivazione con gli esempi recuperati.

Dopo aver consolidato la prima versione basata su `expected_fidelity`, il Dataset potrà essere esteso anche alla metrica `critical_depth`.

Manca anche da fissare la politica sperimentale.

Riferimenti

- [1] N. Quetschlich, L. Burgholzer, R. Wille, MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing, ACM Transactions on Quantum Computing, 2025.
- [2] N. Quetschlich, L. Burgholzer, R. Wille, Predicting Good Quantum Circuit Compilation Options, 2023.
- [3] Munich Quantum Toolkit, MQT Predictor 2.3.0: documentazione e codice sorgente analizzati nell'ambiente di lavoro.